MATH DIRECTED READING PROGRAM / MEETING 3

From Scores to Probabilities and Learning Signals

Self-contained guided reading and problem sheet | Revision 3.0

Aim. Derive a classification loss and its derivatives, then identify where those derivatives enter an output layer and an embedding table. Keep labels, probabilities, computed values, and parameters distinct.

Background and pacing. Exponentials, logarithms, the chain rule, and matrix multiplication. All required probability definitions appear below. Problems 3.1–3.3 form the core; 3.4–3.5 connect it to networks. Extension 3.E is optional.

Using the self-checks. After each numbered problem, try its short self-checks with the worked solution covered. Definitions may stay open. Give a reason, not only a yes/no answer; these checks guide discussion rather than add a timed test.

3.1 Categorical predictions and softmax

SETUP: ONE OUTCOME OUT OF SEVERAL CLASSES

A **classification** task predicts which of $C \geq 2$ possible classes an input belongs to. A random variable $Y \in \{1, \ldots, C\}$ has a **categorical distribution** with probability vector p when

$$\Pr(Y = i) = p_i, \qquad p_i \geq 0, \qquad \sum_{i=1}^{C} p_i = 1.$$

A model may first compute arbitrary real scores $z = (z_1, \ldots, z_C)^\top$, called **logits**. These are not probabilities. The **softmax** map converts them into a strictly positive probability vector:

$$p = \operatorname{softmax}(z), \qquad p_i = \frac{\exp(z_i)}{\sum_{j=1}^{C} \exp(z_j)}.$$

Softmax is a parameterization of categorical probabilities, not the definition of a categorical distribution. The vector z is an output computed from input and parameters, not normally a parameter vector itself. All logarithms in these sheets are natural logarithms.

Worked example. For $z = (\log 2, \log 3, \log 5)^\top$, exponentiation gives $(2, 3, 5)^\top$, and normalization gives $p = (0.2, 0.3, 0.5)^\top$.

Guided questions

(a) Verify $p_i > 0$ and $\sum_i p_i = 1$. If every logit is equal, what distribution results?

(b) Show $p_i / p_j = \exp(z_i - z_j)$. Which class has the greatest probability in the example? Why does ranking logits give the same ordering as ranking probabilities?

(c) Let $\mathbf{1}$ denote the vector of ones. Prove $\operatorname{softmax}(z + c\mathbf{1}) = \operatorname{softmax}(z)$ for any scalar c. Thus logits are not uniquely determined by their probabilities.

(d) Recompute the example after subtracting $\max_j z_j$ from all logits. The exponentials are now at most 1. Why can this equivalent formula be useful when a computer cannot represent arbitrarily large numbers?

Translate back into machine-learning language. Distinguish real-valued logits, normalized probabilities, and a parameter-update rule.

SELF-CHECK 3.1

1. Without evaluating exponentials, compare the distributions from logits $(1001, 1000)$ and $(1, 0)$. Does the raw vector $(1, 0)$ equal that softmax probability vector?
2. Can a finite softmax vector assign probability exactly zero to a class? Distinguish the categorical family from this strictly positive parameterization.

3.2 From observed labels to a loss

REVIEW: LIKELIHOOD AND A ONE-HOT TARGET

For a fixed input x, write $p_{\theta,i}(x) = \Pr_\theta(Y = i \mid x)$. The notation $\mid x$ means that the input is given. If the observed class is the integer y, the model assigns it probability $p_{\theta,y}(x)$.

With the observation fixed and θ varying, this same expression is a **likelihood**. **Maximum likelihood** chooses parameters making the observed outcomes probable. For several fixed inputs, assume their labels are conditionally independent under the model; then the dataset likelihood is

$$\mathcal{L}(\theta) = \prod_{s=1}^{N} p_{\theta,y_s}(x_s).$$

The **negative log-likelihood (NLL)** is $-\log \mathcal{L}$. We use $\mathcal{L}$ for likelihood and L for a loss, not interchangeably. The **one-hot target vector** for class y is $\mathbf{y} = e_y \in \mathbb{R}^C$: its yth entry is 1 and every other entry is 0. The integer y and vector $\mathbf{y}$ encode the same observed label but are different mathematical objects.

For one example, define the **cross-entropy loss**

$$\ell = H(\mathbf{y}, p) := -\sum_{i=1}^{C} \mathbf{y}_i \log p_i.$$

Worked example. With $p = (0.2, 0.3, 0.5)^\top$ and observed class $y = 2$, the target is $\mathbf{y} = (0, 1, 0)^\top$, so the loss is $-\log(0.3) \approx 1.204$. The most probable class and the observed class need not coincide.

Guided questions

(a) Take $-\log$ of the dataset likelihood and express it as a sum. Why does maximizing likelihood give the same optimizers as minimizing this sum? Why does dividing by N preserve those optimizers?

(b) Show that, for a one-hot target, $H(\mathbf{y}, p) = -\log p_y$. This identifies the single-example cross-entropy with NLL in this setting.

(c) Substitute the softmax formula and derive

$$\ell(z, y) = \log\left(\sum_j e^{z_j}\right) - z_y = \log\left(\sum_j e^{z_j}\right) - \mathbf{y}^\top z.$$

(d) If p_y tends to 1, what happens to the loss? If p_y tends to 0, what happens? Explain which probability matters in this formula, even when another class receives a very high probability.

Translate back into machine-learning language. The target vector records one observed outcome; it does not assert that every future outcome for similar inputs is deterministic. The loss rewards probability assigned to the observed label.

> **SELF-CHECK 3.2**
>
> 1. The correct label is class 1. Compare $p = (0.4, 0.6)$ and $p = (0.49, 0.51)$: is the most probable class correct in either case, and which loss is smaller?
> 2. When maximizing likelihood, are the observed label and input changed, or the parameters? Does a one-hot observed label prove the task has no uncertainty?

3.3 Differentiate softmax and cross-entropy

REVIEW: A JACOBIAN AND THE LOG-SUM-EXP FUNCTION

For $p = \mathrm{softmax}(z)$, define the Jacobian J by $J_{ij} = \partial p_i / \partial z_j$. The symbol δ_{ij} is 1 if $i = j$ and 0 otherwise. The diagonal matrix $\mathrm{diag}(p)$ has entries p_i on its diagonal and zeros elsewhere.
Define **log-sum-exp** by $\mathrm{LSE}(z) = \log \sum_j e^{z_j}$. Problem 3.2 rewrites the loss as $\ell = \mathrm{LSE}(z) - \mathbf{y}^\top z$. The target $\mathbf{y}$ stays fixed while differentiating.

Worked derivative of the denominator. Put $S(z) = \sum_k e^{z_k}$. Then $\partial S / \partial z_j = e^{z_j}$ and

$$\frac{\partial \log S}{\partial z_j} = \frac{e^{z_j}}{S}.$$

Use this observation rather than treating the softmax denominator as constant.

Guided questions

(a) Apply the quotient rule to $p_i = e^{z_i}/S$. Handle $i = j$ and $i \neq j$, then combine them to obtain

$$J_{ij} = p_i(\delta_{ij} - p_j), \qquad J = \mathrm{diag}(p) - pp^\top.$$

(b) Differentiate $\ell = \mathrm{LSE}(z) - \mathbf{y}^\top z$. Derive the central identity

$$\boxed{\nabla_z \ell = p - \mathbf{y}.}$$

(c) Derive the same result by the chain rule through p: first find $\partial \ell / \partial p_i$, then sum $\sum_i (\partial \ell / \partial p_i)(\partial p_i / \partial z_j)$. Where is the one-hot target used?

(d) At $p = (0.2, 0.3, 0.5)^\top$ and $y = 2$, calculate $\nabla_z \ell$. In the *hypothetical* update $z^+ = z - \eta \nabla_z \ell$, which logits rise and which fall?

(e) Show $\sum_i (\nabla_z \ell)_i = 0$. Relate this to the fact that adding a common constant to all logits leaves the loss unchanged.

SCOPE OF THE IDENTITY

The formula above is for one example, ordinary softmax with no temperature factor, and unweighted cross-entropy. Averaging N example losses introduces the factor $1/N$ in each example's contribution. Class weights or other loss modifications require redoing the derivative.

Translate back into machine-learning language. The difference $p - \mathbf{y}$ is a derivative *with respect to logits*. The next problem turns it into derivatives with respect to parameters.

SELF-CHECK 3.3

1. For $p = (0.1, 0.6, 0.3)^\top$ and label $y = 3$, find $\nabla_z \ell$. Interpret its signs only as a hypothetical direct logit update, not as a guarantee about a shared network update.
2. If the objective is the *sum* of two identical copies of this loss, what derivative results? What if it is their *average*?

3.4 The output head: where the gradient goes next

SETUP: A REPRESENTATION IS NOT YET A PREDICTION

Let $h \in \mathbb{R}^d$ be a **hidden representation**, a vector computed by earlier layers. An **output head** is the final part of a model that converts this representation into outputs for the task. For our classifier, it consists of

$$z = W_{\text{out}} h + b, \qquad p = \text{softmax}(z), \quad W_{\text{out}} \in \mathbb{R}^{C \times d},\ b \in \mathbb{R}^C.$$

The first map is affine. In machine-learning usage, a **linear layer** often includes a bias, even though it is affine rather than linear in the strict mathematical sense. Here W_{out} and b are parameters; h, z, p are computed values. A layer called a *projection* need not be an orthogonal or idempotent projection.
For $u = Ax + b$ and a scalar loss, the column-gradient chain rule gives $\nabla_A \ell = (\nabla_u \ell) x^\top$, $\nabla_b \ell = \nabla_u \ell$, and $\nabla_x \ell = A^\top \nabla_u \ell$.

Worked forward calculation. Let

$$h = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad W_{\text{out}} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad b = 0.$$

Then $z = (1, 2, -1)^\top$ and $p = (e, e^2, e^{-1})^\top / (e + e^2 + e^{-1})$. The representation has two features; the output has three class scores. Softmax turns those scores into class probabilities.

Vocabulary before the application. A **token** is a discrete symbol processed by a model; in text it may represent a word, a word piece, or a character. A **vocabulary** $\mathcal{V}$ is the fixed set of token types. A **next-token classifier** predicts the next symbol from a supplied sequence prefix.

Guided questions

(a) Let $e = p - \mathbf{y}$. Use the local chain rule to derive $\nabla_{W_{\text{out}}} \ell$, $\nabla_b \ell$, and $\nabla_h \ell$. Check every shape.

(b) With target class $y = 2$ in the worked example, write the three derivatives in terms of p_1, p_2, p_3. Which derivative is passed back to the earlier layers that computed h?

(c) For a next-token classifier, put $C = |\mathcal{V}|$ and let h summarize the given prefix. Explain which coordinate of p the observed next token selects in the loss. Constructing h from the context is a separate task.

ACTUAL TRAINING UPDATES PARAMETERS, NOT EVERY LOGIT IN ISOLATION

For one fixed example with h held fixed, updating only W_{out} and b by gradient descent gives exactly $z^+ = z - \eta(\|h\|_2^2 + 1)(p - \mathbf{y})$. Verify this as an optional calculation. In a full network, h also changes, parameters are shared between examples, and a mini-batch combines their signals. Do not infer that every individual example's correct logit rises after every training step.

Translate back into machine-learning language. Distinguish the output matrix W_{out}, the logit vector z, and the probability vector p using both their shapes and their roles.

SELF-CHECK 3.4

1. For two classes and a two-feature vector, take $h = (1, 0)^\top$, $W_{\text{out}} = 0$, $b = 0$, and $y = 1$. Compute p, the head gradients, and $\nabla_h \ell$. Can head parameters receive a nonzero derivative while the derivative passed to h is zero?
2. With vocabulary size 5 and representation width 2, which shapes belong to W_{out}, h, z, p? Which of those are stored parameters rather than per-input results?

3.5 Embedding lookup is a trainable table

SETUP: TURN A DISCRETE SYMBOL INTO A VECTOR

Recall that a token is a discrete symbol and its vocabulary is the fixed set of allowed types. Let this vocabulary $\mathcal{V}$ have V_{vocab} types, numbered $1, \ldots, V_{\text{vocab}}$.

An **embedding matrix** $E \in \mathbb{R}^{V_{\text{vocab}} \times d}$ stores one trainable row per token type. If the token ID is the integer a and $e_a \in \mathbb{R}^{V_{\text{vocab}}}$ is its one-hot vector, its column-vector **token embedding** is

$$h = E^\top e_a \in \mathbb{R}^d.$$

This multiplication selects row a of E and transposes it. Implementations usually select that row directly: an **embedding lookup**. The token ID is an index, not a continuous trainable input. The width d is the **embedding dimension**.

Worked lookup. For

$$E = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 0 \end{pmatrix}, \qquad a = 3,$$

the embedding is $(2, 0)^\top$. Repeating token ID 3 selects the same row. The lookup alone does not yet distinguish its different surrounding contexts.

Guided questions

(a) Verify the lookup for all three token IDs. Count the trainable scalar entries in a general $V_{\text{vocab}} \times d$ table. Is the one-hot vector itself trainable in this setup?

(b) Suppose later calculations supply $g = \nabla_h \ell \in \mathbb{R}^d$. From $h_j = \sum_i E_{ij} (e_a)_i$, derive

$$\nabla_E \ell = e_a g^\top.$$

Which rows receive zero derivative through this single lookup? Assume the loss uses E nowhere else.

(c) Two occurrences of the same token use the same row and supply derivatives $g^{(1)}$ and $g^{(2)}$. Use the multivariable chain rule to find their combined contribution to $\nabla_E \ell$. Why do the contributions add rather than overwrite each other?

OPTIONAL REVIEW: DOT PRODUCT VERSUS COSINE SIMILARITY

For nonzero vectors u, v, their **cosine similarity** is $u^\top v/(\|u\|_2\|v\|_2)$. It measures directional alignment, while the dot product also depends on lengths. Compute both for rows 1 and 3 of the example. The lookup formula alone does not guarantee meaningful semantic geometry.

Translate back into machine-learning language. "Learning embeddings" means updating the table entries using the task's loss. It does not mean differentiating the integer token IDs. Shared tables can also receive derivatives through other uses, not considered in part (b).

SELF-CHECK 3.5

1. The same token is looked up twice and receives derivatives $(1, 2)^\top$ and $(-1, 3)^\top$. What row derivative is accumulated? Assume no other use of the table.
2. Two identical token IDs occur in different sentences. Does lookup alone give different embeddings? Must a later context-dependent representation remain identical? Distinguish these stages.

Optional extension: soft targets and relative entropy

3.E / REPLACE A ONE-HOT TARGET BY A DISTRIBUTION

Let $r = (r_1, \dots, r_C)^\top$ be a fixed target probability vector. It need not be one-hot; such a target is called a **soft target**. Define

$$H(r, p) = -\sum_i r_i \log p_i, \qquad H(r) = -\sum_i r_i \log r_i,$$

and the **Kullback–Leibler divergence** (**KL divergence**)

$$D_{\mathrm{KL}}(r\|p) = \sum_i r_i \log \frac{r_i}{p_i}.$$

$H(r)$ is the **entropy** of r. Terms with $r_i = 0$ are interpreted as zero. A divergence need not be a distance: KL is generally not symmetric. For the proof below, first assume all r_i, p_i are positive.

Worked comparison. If $r = (1/2, 1/2)$, then $H(r, r) = \log 2$ and $D_{\mathrm{KL}}(r\|r) = 0$. A nonzero optimal cross-entropy does not necessarily indicate a wrong model when the target is not one-hot.

Guided questions

(a) Expand the logarithm of the quotient to prove

$$H(r, p) = H(r) + D_{\mathrm{KL}}(r\|p).$$

Which term is constant when p varies but r does not?

(b) The elementary inequality $\log t \le t - 1$ for $t > 0$ follows by studying $t - 1 - \log t$. Apply it with $t = p_i/r_i$, multiply by r_i, and sum. Prove $D_{\mathrm{KL}}(r\|p) \ge 0$. Identify the equality case.

(c) With $p = \mathrm{softmax}(z)$ and $\sum_i r_i = 1$, rewrite $H(r, p)$ using log-sum-exp, then prove $\nabla_z H(r, p) = p - r$.

(d) If every $r_i > 0$, show that logits $z_i = \log r_i$ represent $p = r$. Now let $r = e_y$ be one-hot. Can any finite logit vector give $p_y = 1$ when $C \ge 2$? Distinguish an infimum of zero loss from an attained zero minimum.

Translate back into machine-learning language. For a fixed target distribution, minimizing cross-entropy is

equivalent to minimizing KL divergence to it. This is not a guarantee that a restricted parameterized model can attain every desired distribution or that an optimizer will find the best parameters.

SELF-CHECK 3.E

1. For target $r = (1/2, 1/2)$, is a cross-entropy minimum of $\log 2$ a failure to match the target? Contrast it with the zero infimum for a one-hot target and finite softmax logits.

Discussion checkpoint and selected hints

READY TO MOVE ON WHEN...

Start from a class label and logits, calculate a loss, derive the signal $p - \mathbf{y}$, and pass it through an affine output head. Explain how a single embedding lookup receives a derivative. A new small numerical example should be manageable without replaying the entire worked derivation.

You may consult definitions and notation. Explain the choices without following a worked solution line by line. If a self-check exposes a gap, return to that problem's setup; optional extensions do not block progress.

CHECK YOUR TRANSLATIONS

Logits are real scores; softmax probabilities are nonnegative and sum to one. A class label is an index, and a one-hot vector is one encoding of that index.

The observed-label NLL equals cross-entropy with a one-hot target. A soft target gives a more general expected negative log probability.

The derivative $p - \mathbf{y}$ is with respect to logits. An output layer converts it to parameter derivatives by the chain rule. Embedding rows are parameters; selected embeddings are computed representations.

Suggested discussion. Start with a three-class example and reach $\nabla_z \ell = p - \mathbf{y}$ without skipping the denominator derivative. Then explain one matrix gradient and one embedding-table gradient.

3.1(c). Factor e^c out of both numerator and denominator. The same calculation justifies subtracting the largest logit.

3.2(c). Use $-\log(e^{z_y}/S) = -z_y + \log S$.

3.3(c). For one-hot $\mathbf{y}$, only $-\log p_y$ contributes. Alternatively use $\partial H/\partial p_i = -\mathbf{y}_i/p_i$ and sum over i.

3.4(a). The output head is the affine layer of Meeting 2, with arriving derivative $e = p - \mathbf{y}$.

3.5(b). Vary one entry E_{ij} and inspect whether row i is selected. The derivative is $(e_a)_i g_j$.

Optional reading: reconnect with standard sources

[1] Prince, *Understanding Deep Learning* (2023). Author's book site. **§5.1**, maximum likelihood; **§5.5**, multiclass classification; **§5.7**, cross-entropy loss. For token embeddings, see **§12.1**, processing text data.

[2] Zhang, Lipton, Li, and Smola, *Dive into Deep Learning*. §4.1, Softmax Regression, especially §§4.1.1–4.1.3, covers scores, normalization, likelihood, and loss derivatives. §22.11, Information Theory, is optional support for 3.E.

[3] PyTorch documentation: mathematical-to-code terminology. `Linear`: an affine map, with weight storage shaped output-by-input. `Embedding`: a vocabulary-by-embedding-dimension table. `CrossEntropyLoss`: its input is logits, not an already-softmaxed probability vector. These links are optional; no programming is required.

Readings supply context, not missing definitions. Exercises and numerical examples are original to this DRP. Prince locators refer to the 2023 first-edition organization; D2L locators refer to its 1.0.3 online edition. Use section titles when editions differ.